SKILLNAVORA · AI TOOLS · MAKE TUTORIAL

Make: Complete Tutorial

Build visual automations in Make (formerly Integromat): scenarios, modules, routers, iterators, webhooks, HTTP and JSON, error handling, templates and a project.

Edition 2026 · skillnavora.in

How to use this guide

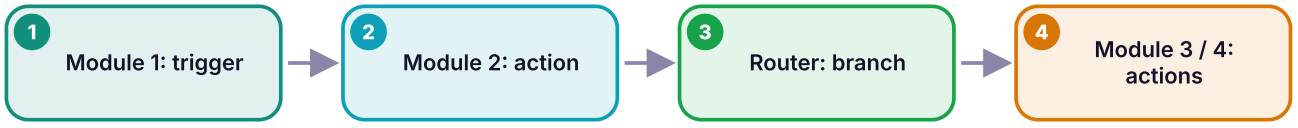
Each class has goals, concepts with Indian examples, key takeaways and a practice exercise, from basic to advanced. Finish with the final project to build something you can show in interviews.

	Class	Level
Class 1	**Make basics: scenarios, modules, operations**	Basic
Class 2	**First scenario: form → Sheet → email**	Basic
Class 3	**Make interface tour and your first canvas habits**	Basic
Class 4	**Flow control: routers, filters, iterators, aggregators**	Intermediate
Class 5	**Webhooks, HTTP and JSON**	Intermediate
Class 6	**Errors, reliability and templates**	Intermediate
Class 7	**Mapping data: functions, dates, text, arrays and JSON**	Intermediate
Class 8	**Real workflows by role**	Intermediate
Class 9	**HTTP, APIs, authentication and pagination**	Intermediate
Class 10	**Reliability: error routes, retries, data stores and locking**	Advanced
Class 11	**AI in Make, agents and responsible use**	Advanced
Class 12	**Cost control, governance and choosing Make vs Zapier vs n8n**	Advanced
Class 13	**Three complete scenarios, step by step**	Advanced
Class 14	**Troubleshooting and FAQ: Make**	Advanced
Class 15	**Practice challenges with solutions: Make**	Advanced
Class 16	**Project and cost control**	Advanced
	Toolkit: templates, make cheat sheet, glossary, 12 interview questions, further reading	

BASIC · CLASS 1

Make basics: scenarios, modules, operations

Understand the canvas and what is billed.



A scenario on the canvas

- **Scenario:** one automation. **Module:** one step (an app action). **Bundle:** one data item flowing through. **Operation:** each module run (billing unit; check current pricing).
- **Scheduling:** run on a timer, or instantly via a webhook trigger.
- Create a scenario at make.com → **Create a new scenario** → click the big **+** to add your first module.

Make vocabulary

Term	Meaning
Scenario	One automation, drawn as connected modules on a canvas
Module	One step (trigger, action, search, tool)
Bundle	One item of data passing through the scenario
Operation	One module run on one bundle; the usual billing unit (check current plan)
Connection	A saved login to an app
Webhook	A URL that starts the scenario when called
Data store	Simple built-in table to keep state between runs
Scheduling	Run on demand, at intervals, or instantly via webhook
Execution history	Log of runs with data in and out
Blueprint	Export of a scenario as a file for backup or sharing

Check before you build: app names, plan limits, prices and menu labels change often. Use this lesson for the method and confirm the current screens and limits in the official docs.

Key takeaways

- Scenario → modules → operations.
- Instant (webhook) or scheduled.
- Data flows as bundles.

Practice exercise: Open Make and add a module from any app; find where history is shown.

BASIC · CLASS 2

First scenario: form → Sheet → email

Build and run a basic scenario.

1. First module: **Google Forms/Typeform → Watch Responses** (or **Webhooks → Custom webhook**). Run once to load sample data.
2. Add **Google Sheets → Add a Row**; map fields by clicking in a box and picking from the data panel.
3. Add **Gmail → Send an Email**; subject "Thanks {{Name}}".
4. Click **Run once**, submit a test; see the data bubbles over each module; then switch **Scheduling** on.

Key takeaways

- Run once to see data.
- Map by picking fields.
- Switch scheduling on.

Practice exercise: Build it and read the execution history for one run.

BASIC · CLASS 3

Make interface tour and your first canvas habits

Get comfortable with the editor and good building habits.

- **Canvas:** drag modules, connect with lines, right-click for options; use **Run once** to test with live data and see bundles at every module.
- **Scenario settings:** schedule, sequential processing, data is confidential, max cycles, auto-commit.
- **Mapping panel:** click a field to insert data from earlier modules; use functions (formatDate, replace, substring, if, ifempty) for transformations.
- **Naming and notes:** rename modules ('Find customer by email') and add notes; future you will be grateful.
- **Folders, teams and organisations:** keep scenarios organised and share with teammates.
- **Templates:** start from a template, then change the connections and test.

Habit	Why
Test each module as you add it	Find mapping errors early
Use filters between modules	Stop unwanted bundles and save operations
Build small, then expand	Easier to debug
Export a blueprint before big changes	Safe backup
Add an error route	No silent failures

Key takeaways

- Run once to inspect data at each step.
- Name modules clearly.
- Back up with a blueprint.

Practice exercise: Open a template, run it once, and write down what data each module outputs.

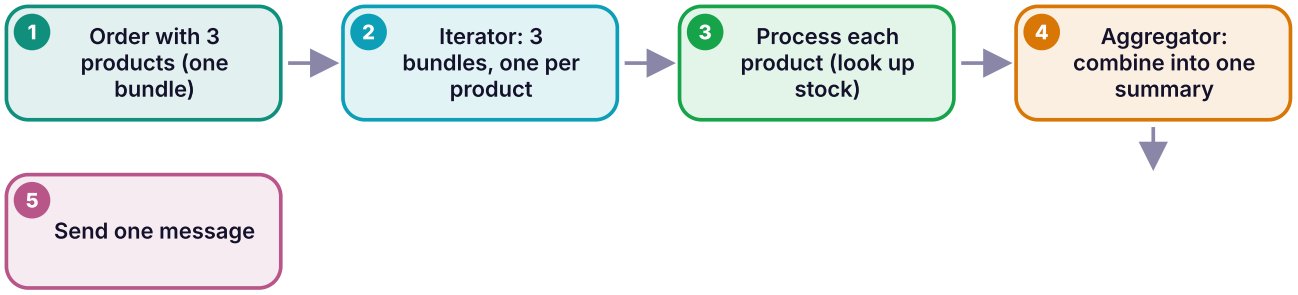
INTERMEDIATE · CLASS 4

Flow control: routers, filters, iterators, aggregators

Branch and loop.

Tool	Use
Router	Split into branches, each with a filter (Hot / Warm / Cold)
Filter	Condition on the line between modules
Iterator	Turn an array (order items) into separate bundles
Aggregator	Combine many bundles into one (summary email, one Sheet row)
Set variable / Tools	Hold values, sleep, compose text
Functions	formatDate, ifempty, split, sum...

Iterator and aggregator in a picture



Split a list, work on each item, then join the results

Use **Iterator** to split an array into items; use an **Aggregator** (array, text, numeric, table) to join results back into one bundle. Without an aggregator, later modules run once per item.

Key takeaways

- Router branches, iterator loops, aggregator combines.
- Filters sit on connections.
- Use functions for formatting.

Practice exercise: Build a router that sends different emails by lead score.

INTERMEDIATE · CLASS 5

Webhooks, HTTP and JSON

Talk to any API.

- **Custom webhook:** gives a URL; click **Redetermine data structure** after sending a sample.
- **HTTP → Make a request:** call any REST API: set method, URL, headers (Authorization: Bearer ...), body type JSON; **Parse response** = yes.
- **JSON module:** create or parse JSON; use the data structure builder.
- **AI modules:** OpenAI, Claude, Gemini apps or the HTTP module to call any model API.

```
HTTP request: POST https://api.example.com/v1/leads
Headers: Authorization: Bearer {{key}}   Body (JSON): {"name": "{{Name}}", "email": "
{{Email}}"}
```

JSON basics for automation builders

```
{
  "order_id": "A-1001",
  "customer": {"name": "Ravi", "phone": "9876543210"},
  "items": [{"sku": "S1", "qty": 2}, {"sku": "S2", "qty": 1}],
  "total": 1499
}
```

- Curly braces hold named fields; square brackets hold lists.
- Use **Parse JSON** with a data structure so fields appear as mappable items.
- For HTTP calls: set method, URL, headers (Authorization), query or body, and parse the response.

Key takeaways

- Webhooks trigger instantly.
- HTTP module calls any API.
- Store keys in connections, not text.

Practice exercise: Call a public API with the HTTP module and map two fields.

INTERMEDIATE · CLASS 6

Errors, reliability and templates

Make scenarios dependable.

- **Error handlers:** right-click a module → **Add error handler**: Ignore, Resume (use a default), Rollback, Break (retry later), or Commit.
- **Incomplete executions:** turn on to keep failed runs and retry.
- **Notifications:** email/Slack alert on errors; name scenarios and use folders.

Template	Flow
Lead router	Webhook → AI classify → Router → Slack / CRM / nurture
Weekly report	Schedule → Sheet rows → Aggregator → AI summary → email
Order sync	Shopify order → Iterator items → Sheet rows

Error handling directives

Directive	What it does	Use for
Ignore	Skip the failed bundle and continue	Non-critical steps
Resume	Substitute a default value and continue	Optional data
Commit	Stop and keep what was already done	Partial success is acceptable
Rollback	Stop and undo changes where the apps support it	Transactions that must be all-or-nothing
Break	Store as an incomplete execution to retry later	Temporary failures like rate limits

Key takeaways

- Add error handlers on important modules.
- Keep incomplete executions.
- Alert on failures.

Practice exercise: Add a Break handler and an alert to your scenario and force one failure.

INTERMEDIATE · CLASS 7

Mapping data: functions, dates, text, arrays and JSON

Transform data without code.

Task	Function / module	Example
Join first and last name	Text: concatenation or a variable	Ravi + Kumar = Ravi Kumar
Replace text	replace(text; old; new)	Remove spaces from a phone number
Format date	formatDate(date; format; timezone)	04 Nov 2026 10:30 in Asia/Kolkata
Add days	addDays(date; n)	Follow-up date
Default value	ifempty(value; default)	Use 'Unknown' when blank
Conditional text	if(condition; a; b)	Hot if budget > 50,000 else Warm
Get item from array	get(array; index) or first()	First product name
Sum numbers	sum(array)	Order total
Create JSON	Create JSON / JSON module	Payload to send to another API
Regex	Text parser: match pattern	Extract order number from text

Tips: watch data types (text vs number vs date), time zones, and array positions (Make arrays start at 1).

Key takeaways

- Know your types: text, number, date, array.
- Use ifempty and if to avoid blanks and errors.
- Convert time zones deliberately.

Practice exercise: Write the mapping expressions to build a clean customer record from a messy form entry.

INTERMEDIATE · CLASS 8

Real workflows by role

Copy practical scenarios and adapt them.

Role	Scenario	Modules in order
Product manager	Feedback to backlog	Watch rows (Sheet) → HTTP or AI module to classify → Notion/Jira create item → Slack message
Marketing	Lead capture and nurture	Webhook (form) → Router (hot/cold) → CRM create → email sequence start → data store log
Support	Ticket categorisation	Watch emails → AI categorise → Router → helpdesk create ticket → acknowledgement email
Finance	Invoice to sheet	Watch email attachments → OCR or AI extract → Sheet add row → Slack approval message
E-commerce	Order and inventory sync	Webhook (new order) → Iterator (line items) → update stock sheet → aggregator → confirmation message
Operations	Daily report	Schedule 8 am → fetch data (HTTP) → aggregate → build HTML → email
HR	Onboarding	Watch new employee row → create accounts via HTTP → send checklist → calendar invites

Start with the simplest version, add routers for exceptions and error handlers last.

Key takeaways

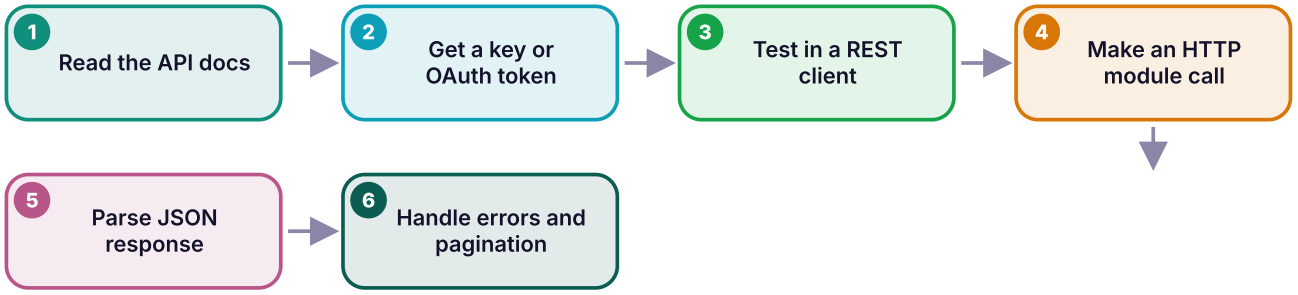
- Pick workflows with clear triggers.
- Add routers for exceptions.
- Document each scenario.

Practice exercise: Choose one scenario and draw its module map with error routes.

INTERMEDIATE · CLASS 9

HTTP, APIs, authentication and pagination

Connect to any service, even without a ready-made app.



Connecting to a custom API

- **Authentication:** API key in a header, bearer token, basic auth, or OAuth 2.0 (use Make's connection support). Store secrets in connections or scenario variables, never in plain notes.
- **Pagination:** many APIs return pages; loop with a repeater or 'next page' token until done.
- **Rate limits:** add sleep modules or use sequential processing; respect HTTP 429 responses.
- **Webhook security:** add a secret in the URL or header and check it in a filter.
- **Idempotency:** include unique IDs to prevent duplicate orders or payments if a run is retried.

HTTP status	Meaning	Action
200 / 201	Success / created	Continue
400	Bad request	Fix payload
401 / 403	Not allowed	Check key or scope
404	Not found	Check ID or URL
429	Too many requests	Wait and retry
500+	Server error	Retry later; alert if repeated

Key takeaways

- Use connections for secrets.
- Handle pagination and rate limits.
- Include unique IDs to avoid duplicates.

Practice exercise: Call a public API (for example a weather or currency API) from Make and save results to a sheet.

ADVANCED · CLASS 10

Reliability: error routes, retries, data stores and locking

Make scenarios resilient.

Problem	Design
Temporary outage	Break directive with automatic retries; alert if retries exhausted
Bad data in one bundle	Error route that logs the bundle to a sheet and notifies the owner; continue others
Duplicate runs	Store processed IDs in a data store and skip if seen
Order of processing matters	Enable sequential processing; use queues
Two scenarios editing the same record	Use a lock flag in a data store or process via one scenario
Large volumes	Process in batches; schedule off-peak; split scenarios
Monitoring	Email or Slack alerts on error, daily summary of runs and failures

Incomplete executions

When Break is used, failed runs wait in **Incomplete executions**. Review daily, fix the cause, and run them again.

Key takeaways

- Create an error route for every important scenario.
- Use data stores to remember what is done.
- Review incomplete executions regularly.

Practice exercise: Add error handling and duplicate protection to a scenario that creates CRM contacts.

ADVANCED · CLASS 11

AI in Make, agents and responsible use

Add AI steps with control.

- **AI modules:** classify, extract, summarise, translate and draft using providers such as OpenAI, Anthropic and Gemini through ready modules or HTTP calls; Make also offers its own AI features and agent-style tools (confirm current names in the docs).
- **Pattern:** collect input → AI with structured JSON output → parse → validate (router with filters) → act or send to human → log.
- **Cost:** each AI call adds provider tokens and one or more operations; monitor both.
- **Safety:** avoid sending sensitive data unless allowed; ask AI to return 'unknown' when unsure; require human approval for customer-facing or money-related actions.
- **Prompt versions:** keep prompts in a data store or note with a version and test cases.

Use case	Prompt output	Validation
Lead scoring	{score: 1-10, reason}	Score is a number 1-10
Invoice extraction	{vendor, date, amount, tax}	Amount > 0, date valid, totals match
Feedback tagging	{topic, sentiment}	Topic in allowed list

Key takeaways

- Ask for JSON and validate it.
- Monitor provider cost and operations.
- Human approval for risky actions.

Practice exercise: Design the router and validation logic for an AI invoice-extraction scenario.

ADVANCED · CLASS 12

Cost control, governance and choosing Make vs Zapier vs n8n

Run automations at scale and pick the right tool.

Lever	How it reduces operations
Filter early	Stop unwanted bundles before costly modules
Aggregate	Send one message or API call instead of many
Use schedules	Poll less often when real-time is not needed
Webhooks over polling	Instant and cheaper than frequent polling
Avoid unnecessary searches	Pass IDs instead of searching again
Clean up	Turn off test scenarios; review top consumers monthly

Governance

- Use an organisation account with roles; avoid personal connections.
- Keep an inventory: purpose, owner, data used, apps, last review.
- Peer review changes to critical scenarios; export blueprints as backups.
- Define who gets alerts and who fixes.

Choose	When
Make	Visual, data-heavy workflows with routers, iterators and detailed control
Zapier	Fast simple automations and the widest app coverage
n8n	Technical teams, self-hosting, privacy, custom code and scale
Code	Core product logic and heavy workloads

Key takeaways

- Filter, aggregate and schedule wisely.
- Inventory and review automations.
- Pick the tool by logic complexity and data sensitivity.

Practice exercise: Estimate operations for your top scenarios and choose two cost-saving changes.

ADVANCED · CLASS 13

Three complete scenarios, step by step

Reproduce detailed builds.

Scenario 1: Webhook lead intake with scoring

1. Custom webhook receives form data.
2. Set variables: clean phone, name, normalised email.
3. Search CRM by email; router: update or create.
4. AI module scores 1 to 10 with reason (JSON); parse JSON.
5. Router: score ≥ 8 → Slack alert + task for sales; else → add to nurture sequence.
6. Data store: save lead ID to prevent duplicates; error route emails the owner.

Scenario 2: Order with many items to inventory and confirmation

1. Webhook receives order JSON; parse JSON.
2. Iterator over items; for each, update stock in sheet or database.
3. Aggregator builds a summary text; send confirmation by email or WhatsApp.
4. Router: if any item below threshold → alert the purchasing channel.
5. Error handler: commit what succeeded and log failures to a sheet.

Scenario 3: Daily operations digest

1. Scheduler at 8:00 on weekdays.
2. HTTP modules fetch orders, tickets and failures (with pagination).
3. Aggregate counts and top issues; AI writes a 5-line summary.
4. Build an HTML email with a table; send to the leadership list; archive in Drive.

Key takeaways

- Parse, validate, route, act, log.
- Protect against duplicates and failures.
- Summaries help humans stay in control.

Practice exercise: Build one scenario with test data and run it 10 times, including a failure case.

ADVANCED · CLASS 14

Troubleshooting and FAQ: Make

Fix common problems quickly.

Problem	Likely cause	Fix
Scenario does not start	Scheduling off, webhook not hit, or the connection expired	Turn on, check schedule, test webhook URL, reauthorise connection
Module returns no bundles	Filter or search found nothing	Run once and inspect; adjust search criteria
Mapped value is wrong type	Text vs number vs date	Use parseNumber, parseDate, toString functions
Runs many times per item	Missing aggregator	Add an aggregator after the iterator
Rate limit (429)	API limits	Add sleep or sequential processing; retry with Break
Operations used up	Frequent polling or many modules	Use webhooks, filters and aggregation

Debug routine: use Run once, click each module bubble to see input and output bundles, and fix the first module whose output is wrong.

Key takeaways

- Inspect bundles at each module.
- Control types and aggregation.
- Use webhooks and filters to save operations.

Practice exercise: Intentionally break a scenario three ways and fix each.

ADVANCED · CLASS 15

Practice challenges with solutions: Make

Test yourself on realistic tasks.

Challenge 1

Receive orders by webhook and email a single summary per order, even with many items.

Solution: Webhook → Parse JSON → Iterator (items) → Text aggregator (lines) → Email with the combined text.

Challenge 2

Fetch 500 records from a paginated API into a sheet.

Solution: Repeater or loop with page token → HTTP GET → Parse JSON → Iterator → Sheet add row; stop when next token is empty.

Challenge 3

Skip records already processed.

Solution: Data store: search by ID → Filter (not found) → process → add ID to the store.

Challenge 4

Alert if any run fails.

Solution: Add an error handler route with Break plus a Slack module with scenario name, bundle info and a link to the execution.

Key takeaways

- Combine iterator, aggregator, store and error routes.
- Think about pagination and retries.
- Always add alerts.

Practice exercise: Create your own three challenges with solutions.

ADVANCED · CLASS 16

Project and cost control

Ship a scenario you can show.

Project: AI lead router

Website form (webhook) → AI classifies and summarises → Router: Hot → Slack + calendar link; Warm → nurture email; Cold → Sheet. Weekly aggregator report. Error handlers + alerts.

- **Cost:** operations = modules × runs; filter early, avoid polling when a webhook exists, combine steps.
- **Security:** private connections, minimal personal data, rotate API keys.

Key takeaways

- Filter early to save operations.
- Prefer webhooks to polling.
- Document owner and alerts.

Practice exercise: Complete the AI lead router and estimate monthly operations.

TOOLKIT

Templates

Make project brief

Field	Fill in
Trigger (webhook / schedule)	
Modules	
Router branches	
Error handling	
Notifications + owner	
Operations per month	

TOOLKIT

Make cheat sheet

Term / tool	Meaning / use
Scenario	One automation
Module	One step
Bundle	One data item
Operation	One module run (billing)
Router	Branching
Iterator / Aggregator	Split / combine arrays
Error handler	Defines what happens on failure
Split and join	Iterator then aggregator
Error plan	Route, directive, alert, review incomplete runs
Cost rule	Filter, aggregate, webhook, schedule
AI rule	JSON out, validate, human for risky

Glossary

Router	Splits flow into multiple routes with filters
Iterator	Splits an array into individual bundles
Aggregator	Combines bundles into one
Data store	Built-in key-value table
Incomplete execution	A run saved for retry after a Break
Blueprint	Exported scenario file
Sequential processing	Process bundles one at a time in order
Repeater	Generates a series of bundles, useful for pagination
Parse JSON	Turn JSON text into mappable fields

INTERVIEW PREP

12 interview questions with answers

Q1. What is Make?
A visual no-code automation builder with modules connected into scenarios.

Q2. Iterator vs aggregator?
Iterator splits an array into bundles; aggregator merges bundles into one.

Q3. How do you handle errors?
Add error handlers (ignore, resume, rollback, break), keep incomplete executions, and alert.

Q4. When choose Make over Zapier?
For branching, loops, API calls and cost-efficiency; Zapier for the widest app library and simplicity.

Q5. What is the difference between an operation and a bundle in Make?
A bundle is one item of data; an operation is one module processing one bundle. Operations are typically what counts toward the plan (check current rules).

Q6. When do you use an iterator and an aggregator?
Use an iterator to split an array into individual items and an aggregator to combine results back into a single bundle.

Q7. How do you handle errors in Make?
Add error routes with directives: ignore, resume, commit, rollback or break, and review incomplete executions; alert owners.

Q8. How do you prevent duplicate processing?
Store processed IDs in a data store, search before creating, and use idempotency keys when calling APIs.

Q9. How would you connect an app with no module?
Use the HTTP module with the API's authentication, parse the JSON response, and handle pagination and rate limits.

Q10. How do you secure webhooks?
Use a secret in the URL or header, validate it, restrict logging of sensitive data, and rotate if exposed.

Q11. How do you reduce operations usage?
Filter early, aggregate, use webhooks instead of polling, schedule wisely and remove unused scenarios.

Q12. Make or Zapier?
Make for visual, data-heavy, multi-branch scenarios; Zapier for fast simple automations and the largest app library; choose after testing a real workflow in each.

LEARN MORE

Further reading and tools

- Make Academy ↗
- Make Help Center ↗
- Make templates ↗
- Make community ↗
- Make: functions reference ↗
- Make: error handling ↗

© SkillNavora · skillnavora.in · Part of the AI Product Management and GenAI Launchpad programs. Practise with the free AI Tutor, AI PM Toolkit and Practice Lab on skillnavora.in.